

30-Day FreeRTOS Course for ESP32 Using ESP-IDF

(Day 15)

“Event Groups”



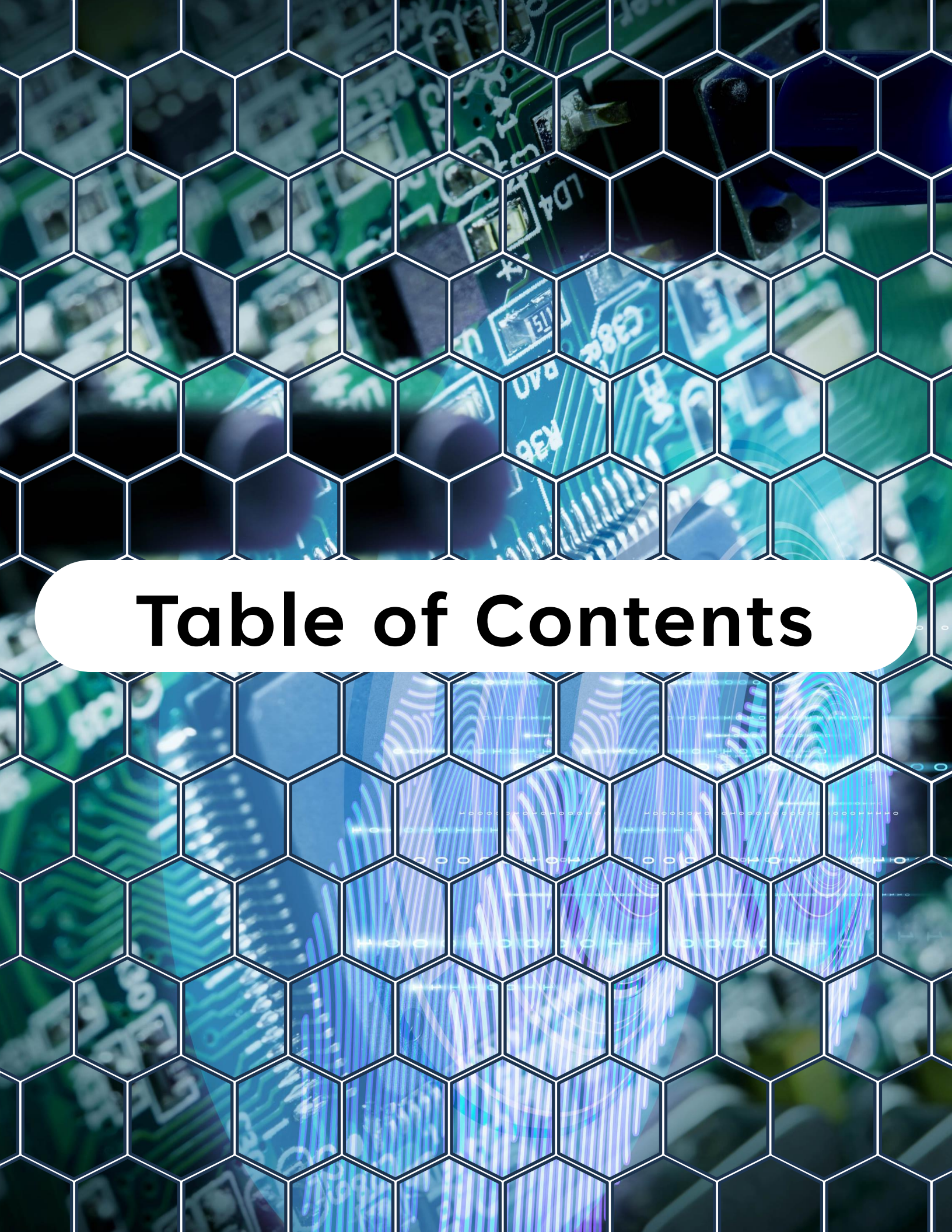
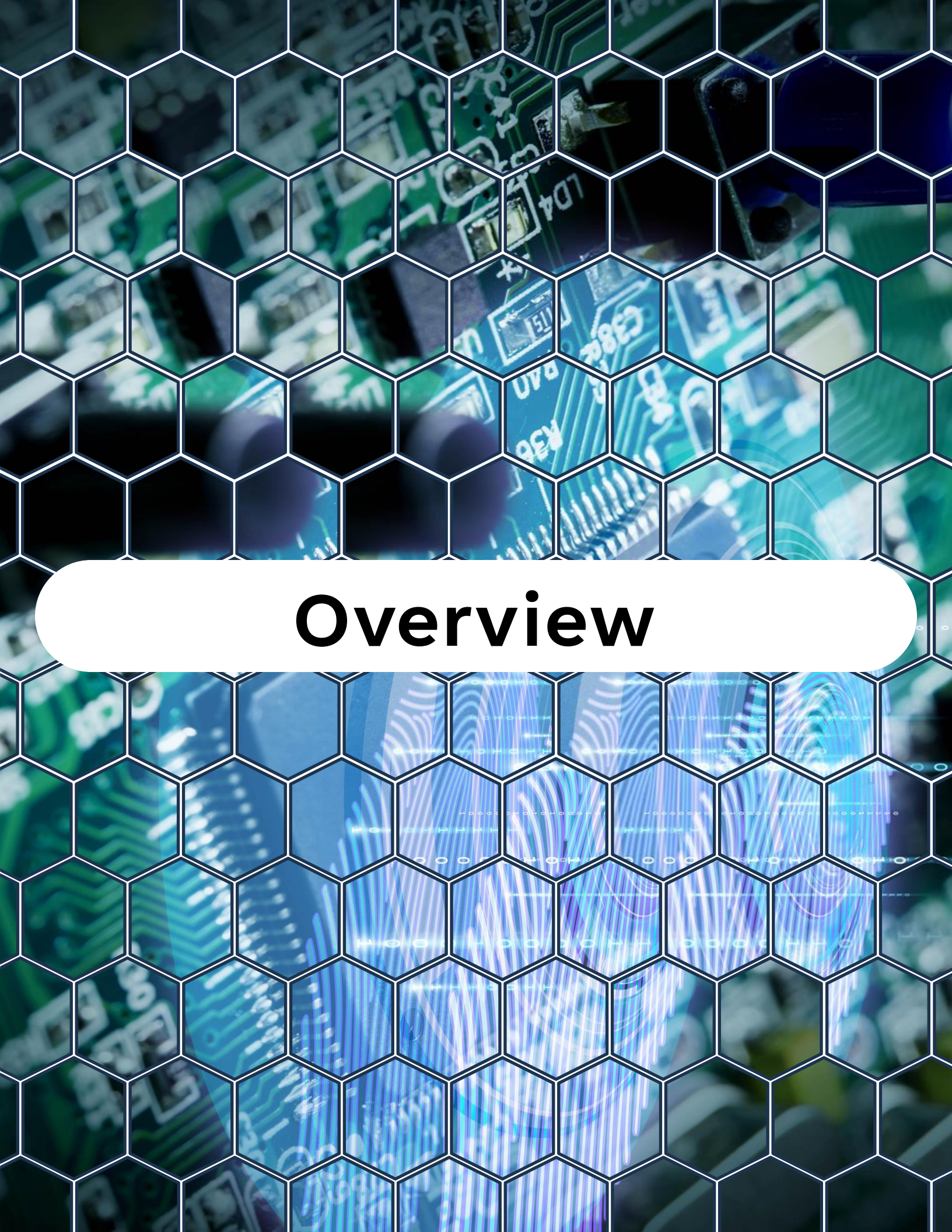


Table of Contents

Table of Contents

1. Overview
2. What Is an Event Group?
3. Event Group Functions
4. Use Cases for Event Groups
5. Example – Multi-Task Synchronization with Event Groups
6. Best Practices
7. Summary
8. Challenge for Today



Overview

Overview

On **Day 15**, you'll learn:

- What **event groups** are in FreeRTOS
- How event groups differ from semaphores and queues
- How to create, set, clear, and wait on event bits
- Using **single-bit** vs **multi-bit** synchronization
- A practical ESP32 example with **multiple tasks waiting on different events**
- Best practices for event group usage

By the end of this lesson, you'll know how to use **event groups** to synchronize multiple tasks using bit flags.



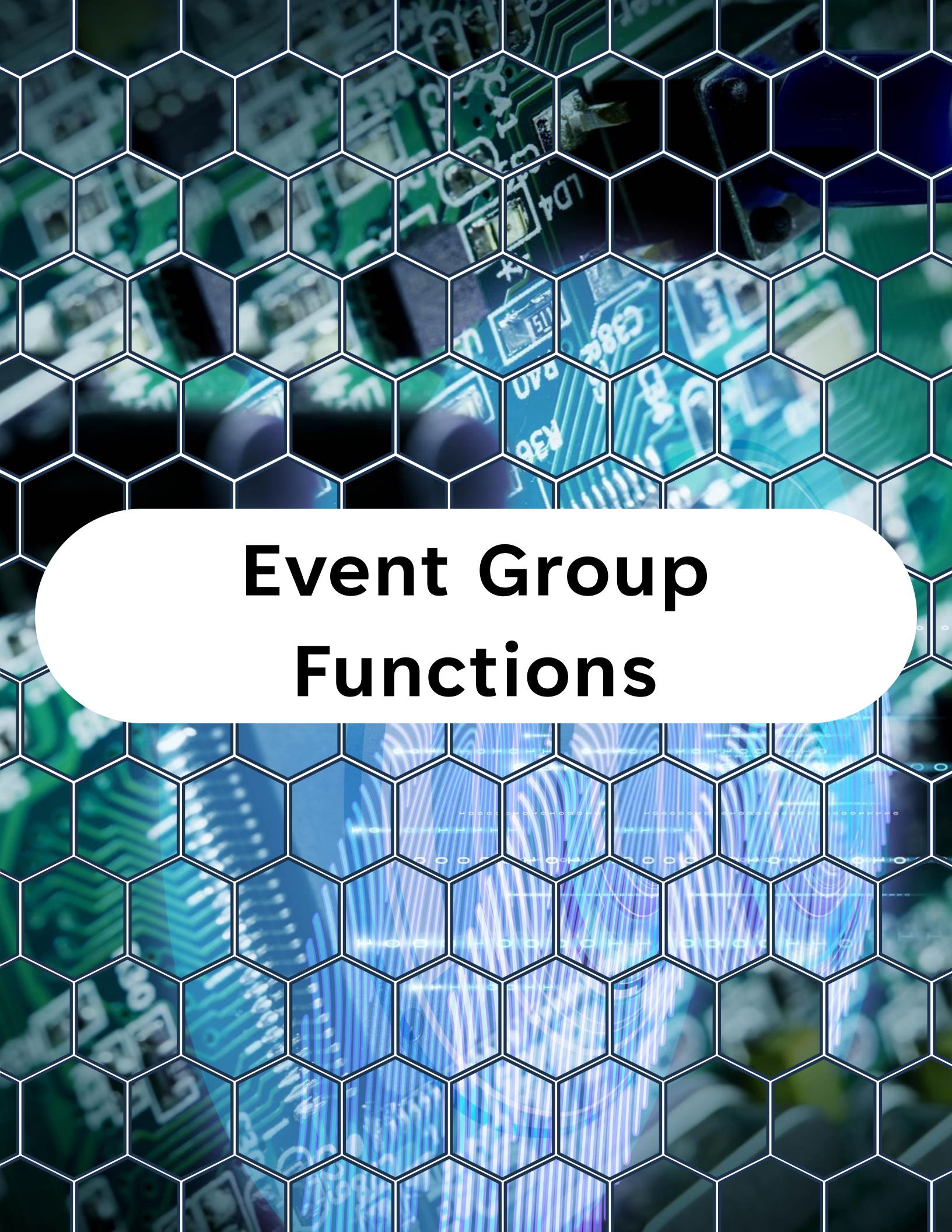
What Is an Event Group?

What Is an Event Group?

An **event group** is a collection of bits used for **task synchronization**:

- Each bit represents an **event flag**.
- Tasks can **set**, **clear**, or **wait for one or more bits**.
- Event groups allow **multi-event synchronization** (tasks can wait on several conditions at once).

 Think of event groups as a **set of binary flags** for coordinating multiple tasks.



Event Group Functions

Event Group Functions

Creating an Event Group



```
1 EventGroupHandle_t xEventGroupCreate(void);
```

Setting and Clearing Bits



```
1 xEventGroupSetBits(EventGroupHandle_t xEventGroup,  
2                     EventBits_t uxBitsToSet);  
3 xEventGroupClearBits(EventGroupHandle_t xEventGroup,  
4                      EventBits_t uxBitsToClear);
```

Waiting for Bits



```
1 xEventGroupWaitBits(EventGroupHandle_t xEventGroup,  
2                     EventBits_t uxBitsToWaitFor,  
3                     BaseType_t xClearOnExit,  
4                     BaseType_t xWaitForAllBits,  
5                     TickType_t xTicksToWait);
```

- `xClearOnExit`: If `pdTRUE`, bits are cleared when a task unblocks.
- `xWaitForAllBits`: If `pdTRUE`, the task waits

2. Event Group Functions

Waiting for Bits



```
1 xEventGroupWaitBits(EventGroupHandle_t xEventGroup,  
2                     EventBits_t uxBitsToWaitFor,  
3                     BaseType_t xClearOnExit,  
4                     BaseType_t xWaitForAllBits,  
5                     TickType_t xTicksToWait);
```

- `xClearOnExit`: If `pdTRUE`, bits are cleared when a task unblocks.
- `xWaitForAllBits`: If `pdTRUE`, the task waits until all bits are set; otherwise, any one bit is enough.

ISR-safe version for setting bits:



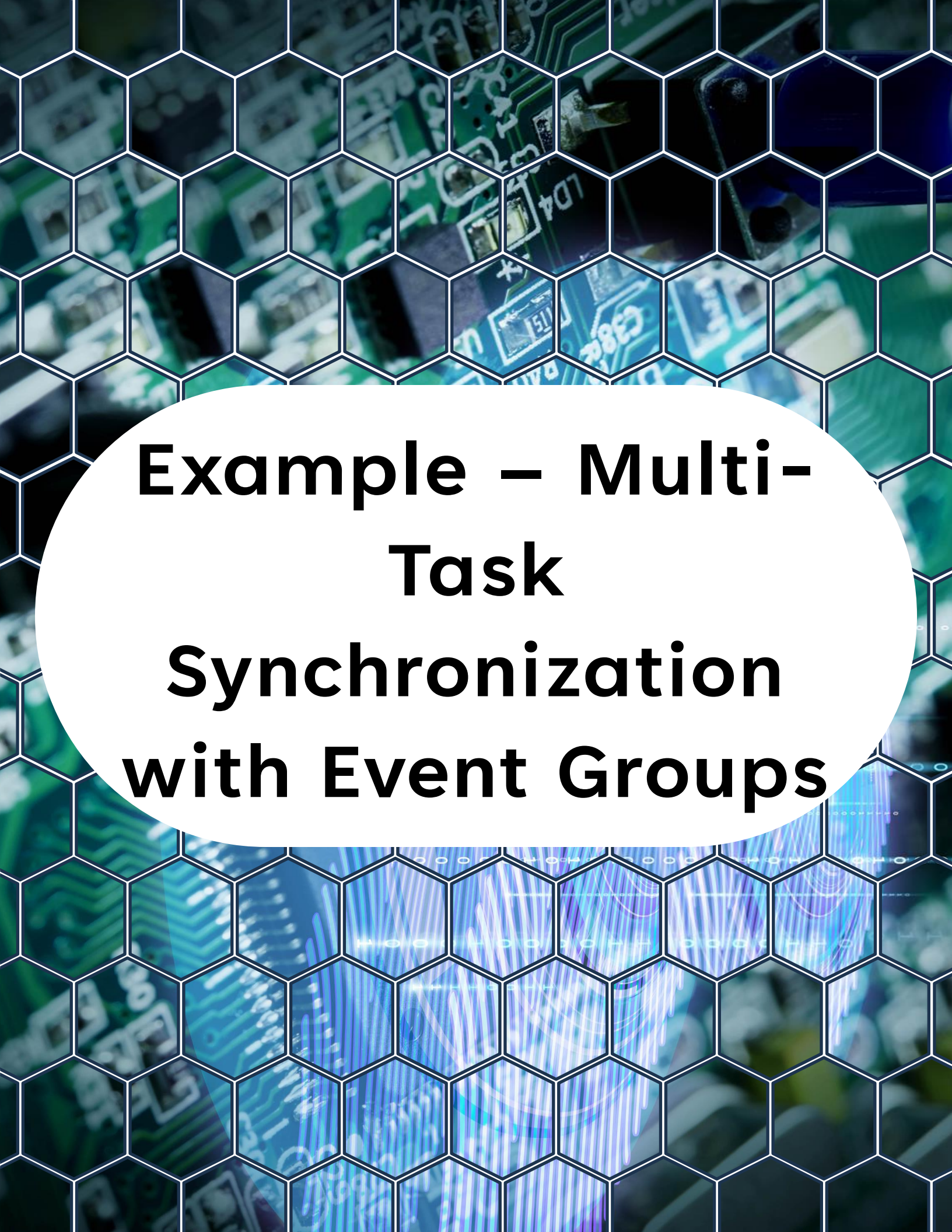
```
1 xEventGroupSetBitsFromISR(...);
```




Use Cases for Event Groups

Use Cases for Event Groups

1. **Multiple tasks waiting on the same event**
2. **One task waiting on multiple conditions**
3. **System state tracking** (e.g., Wi-Fi connected, sensor ready, UART initialized)
4. **Complex synchronization** where semaphores/queues aren't flexible enough



Example – Multi- Task Synchronization with Event Groups

Example – Multi-Task Synchronization with Event Groups

```
1 #include <stdio.h>
2 #include "freertos/FreeRTOS.h"
3 #include "freertos/task.h"
4 #include "freertos/event_groups.h"
5 #include "esp_log.h"
6
7 #define TAG "DAY15"
8
9 // Define event bits
10 #define BIT_TASK_A    (1 << 0)
11 #define BIT_TASK_B    (1 << 1)
12
13 // Declare the event group handle for synchronizing tasks
14 EventGroupHandle_t sync_event_group;
15
16 // Task A simulates some work and then sets its corresponding
17 // event bit in the event group to signal completion.
18 void taskA(void *pvParameter) {
19     vTaskDelay(pdMS_TO_TICKS(2000)); // simulate work
20     ESP_LOGI(TAG, "Task A finished work, setting BIT_TASK_A");
21     xEventGroupSetBits(sync_event_group, BIT_TASK_A);
22     vTaskDelete(NULL);
23 }
24
25 // Task B simulates some work and then sets its corresponding
26 // event bit in the event group to signal completion.
27 void taskB(void *pvParameter) {
28     vTaskDelay(pdMS_TO_TICKS(4000)); // simulate work
29     ESP_LOGI(TAG, "Task B finished work, setting BIT_TASK_B");
30     xEventGroupSetBits(sync_event_group, BIT_TASK_B);
31     vTaskDelete(NULL);
32 }
33
34 // Coordinator task that waits for both Task A and Task B
35 // to finish before proceeding
36 void coordinator_task(void *pvParameter) {
37     ESP_LOGI(TAG, "Coordinator waiting for Task A and B...");
38     EventBits_t bits = xEventGroupWaitBits(
39         sync_event_group,
40         BIT_TASK_A | BIT_TASK_B,    // Wait for both
```


Example – Multi-Task Synchronization with Event Groups

```
33
34 // Coordinator task that waits for both Task A and Task B
35 // to finish before proceeding
36 void coordinator_task(void *pvParameter) {
37     ESP_LOGI(TAG, "Coordinator waiting for Task A and B...");
38     EventBits_t bits = xEventGroupWaitBits(
39         sync_event_group,
40         BIT_TASK_A | BIT_TASK_B,    // Wait for both
41         pdTRUE,                      // Clear bits on exit
42         pdTRUE,                      // Wait for ALL bits
43         portMAX_DELAY               // Block indefinitely
44     );
45
46     if ((bits & (BIT_TASK_A | BIT_TASK_B)) == (BIT_TASK_A | BIT_TASK_B)) {
47         ESP_LOGI(TAG, "Both Task A and Task B finished, Coordinator proceeds!");
48     }
49
50     vTaskDelete(NULL);
51 }
52
53 // Main application entry point
54 void app_main() {
55     // Create event group
56     sync_event_group = xEventGroupCreate();
57
58     // Create tasks
59     xTaskCreate(taskA, "TaskA", 2048, NULL, 5, NULL);
60     xTaskCreate(taskB, "TaskB", 2048, NULL, 5, NULL);
61     xTaskCreate(coordinator_task, "Coordinator", 2048, NULL, 6, NULL);
62 }
```

Example – Multi-Task Synchronization with Event Groups

Expected Behavior

- Task A finishes after 2 seconds → sets **BIT_TASK_A**.
- Task B finishes after 4 seconds → sets **BIT_TASK_B**.

The **Coordinator task** waits until **both bits are set**, then proceeds.

Serial output:

I (2000) DAY15: Task A finished work, setting BIT_TASK_A

I (4000) DAY15: Task B finished work, setting BIT_TASK_B

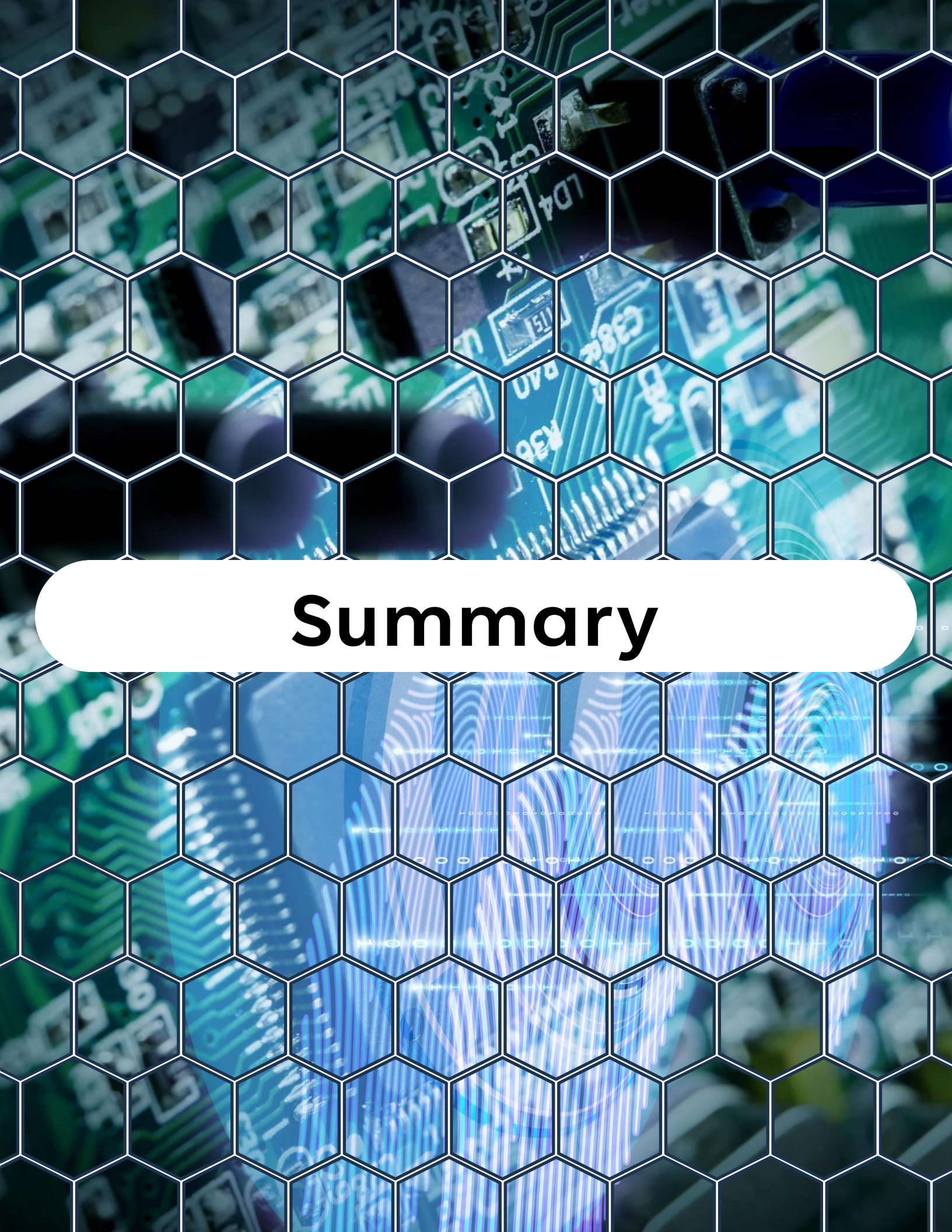
I (4000) DAY15: Both Task A and Task B finished, Coordinator proceeds!



Best Practices

Best Practices

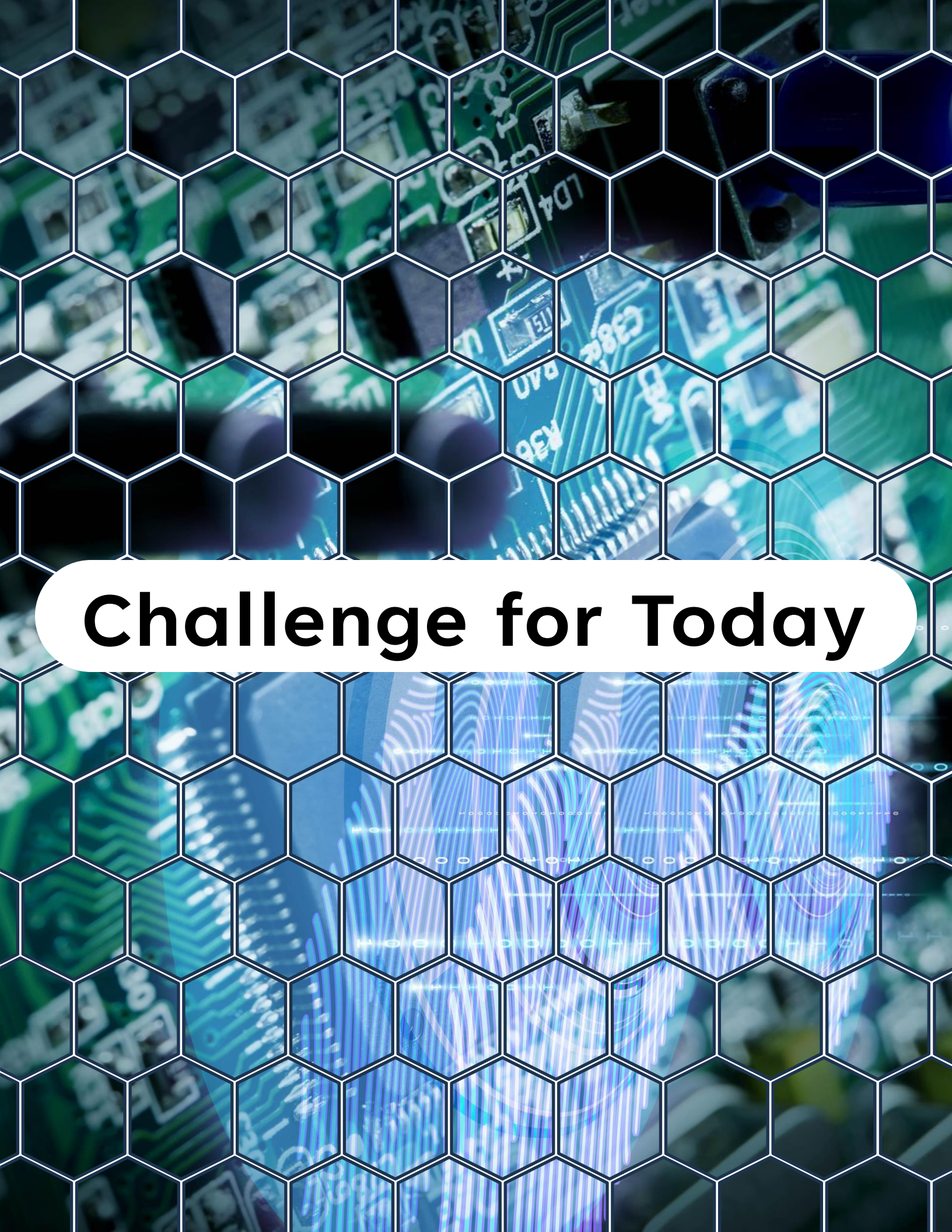
- ✓ Use **event groups** for **synchronization**, not data transfer (use queues for data).
- ✓ Keep event groups small (32 bits max).
- ✓ Use `xWaitForAllBits` carefully — if one bit never arrives, the task blocks forever.
- ✓ For ISR → task signaling, use `xEventGroupSetBitsFromISR()`.
- ✓ Use logging or debugging tools to track system state when multiple events are involved.



Summary

Summary

- Event groups are **bit flags** for flexible task synchronization.
- They let tasks **wait on one or more events simultaneously**.
- Use `xEventGroupWaitBits()` with options to wait for **any** or **all** events.
- Perfect for **system coordination** when multiple conditions must align.



Challenge for Today

Challenge for Today



Extend today's example by:

1. Adding a **third task** that sets `BIT_TASK_C` after 6 seconds.
2. Modify the coordinator to proceed when **any two out of three tasks** finish.
3. Experiment with `xWaitForAllBits = pdFALSE` (any bit) vs `pdTRUE` (all bits).

"Thanks for watching!

If you enjoyed this video,

*make sure to hit the like button and
subscribe to stay updated with my latest
content.*

*Don't forget to check out my other
videos for more tips and tutorials on
Embedded C, Python, hardware designs,
etc. Keep exploring, keep learning, and
I'll see you in the next video!"*



<https://www.linkedin.com/in/yamil-garcia>

<https://www.youtube.com/@LearningByTutorials>

<https://github.com/god233012yamil>